TECHNICAL CASE · FOR REVIEW

Verifiable Guidance Layer

A verifiable knowledge layer between a standards body's guidance and the AI systems that answer questions about it. A proof of concept.

A clause can remain word-for-word identical while the correct answer changes, because a definition, exception, or ruling elsewhere has changed — so truth has to live in a versioned graph, not in a prompt, and every answer has to be checked against it before a person sees it.

EVERYTHING, ONE PLACE

ericediger.com/vgl/review

The video, this document, the walkthrough and the repository.

STEP THROUGH IT · 10 MIN

ericediger.com/vgl

Nine sections, forty-seven steps of recorded output.

RUN IT · ONE COMMAND

github.com/ericediger/verifiable-guidance-layer

Docker only; no API keys needed for the recorded path.

Eric Ediger · November 2026 · The corpus is entirely fictional — a *Global Standards Board* shaped like a real standards body, with nothing copied from, or claimed about, any real standard. Every number in this document is a statement about that corpus.

The brief, and where this submission answers it

A global body publishes the authoritative interpretation of regulatory standards for a highly regulated industry: guidance documents, technical standards, interpretive rulings, compliance frameworks — hundreds of thousands of pages, across jurisdictions, continuously updated. It is building AI tools so that a compliance officer can ask what a standard requires, how it interacts with other standards, and whether a proposed action is consistent with current guidance.

The problem it keeps running into: the AI gives confident answers that are wrong in ways that matter. Not obviously wrong. The standard has been updated and the clause now reads differently. The standard interacts with a second standard in a way that changes the conclusion. An interpretive ruling carves out an exception the AI is unaware of. The officer does not catch it; the advice goes to a client; the client acts on it.

The brief adds four facts about the knowledge itself: the guidance is not flat — standards reference standards, rulings depend on definitions established elsewhere, jurisdictions branch the answer, and updates cascade; experienced practitioners carry applied knowledge that has no authoritative home; the ground moves continuously; and what is needed is a layer between the guidance and the AI systems that professionals can trust and agents can use with confidence — not just searchable but reliable, not just retrievable but verifiable.

Three deliverables. 01 — a high-level architecture for a proof of concept, with a diagram, and the path from 0 → 1: the first working version, what done means, which decisions constrain or enable what comes next. 02 — something concrete and real. 03 — every significant decision: what was chosen, what was considered and rejected, what would be done differently with more time; especially the decisions that were not obvious.

WHAT THE BRIEF NAMES → THE MECHANISM → WHERE TO SEE IT

THE BRIEF	THE LAYER'S ANSWER	SEE IT
The standard has been updated; the clause now reads differently	The version in force is selected by date; the superseded one is history, never evidence, and V3 removes any citation to it	Walkthrough §4 · cases G1, G7
A second standard changes the conclusion	Edges link concepts, so an unchanged provision resolves against the *definition in force*	§7 · G3, G11a, G11b
A ruling carves out an exception the AI is unaware of	Held: it is in the evidence and V4 forces it to surface. Not held: a *known* gap → REVIEW, with the gap named	§6 · G4a → G4b
Jurisdictions branch the answer	Each jurisdiction's own adoption date selects the version; a pending adoption is flagged	§5 · G2
Updates cascade into what references them	The graph is walked backwards before publication; issued client answers expire with a date and a reason	§7 · HPA-77
Practitioner knowledge has no authoritative home	A third authority tier: attached to clauses, surfaced with its label, never able to ground a claim (V6)	§5 · PN-001
The ground moves continuously	Releases as watermarks: every answer is pinned, and an earlier release's evidence reproduces exactly	§6 · Release 1 still REVIEW
Trusted by professionals; usable by agents	Deterministic verification before anything is shown; four outcomes; the same JSON for a screen and a tool call	§3 · `/api/ask`

The problem, reframed — and four commitments

The brief describes an assistant that gives confident answers that are wrong in ways that matter: a clause that has since been amended, a second standard that changes the conclusion, a ruling that carves out an exception.

None of these is a reasoning failure. They are knowledge-structure failures: the model answers from a flat, unversioned, relationship-blind pile of text, and nothing checks the answer. Better retrieval over the clause's own text cannot see a definition that moved, an adoption not yet made, or a ruling never held.

So the design starts from four commitments:

1. **Truth lives in a deterministic, versioned, relationship-aware layer.**
2. **Language models handle language** — understanding the question, composing the answer — and nothing else.
3. **Nothing reaches a person unchecked.** Every probabilistic step sits upstream of a deterministic one: a model may narrow a search or add a flag, never add knowledge or clear a check.
4. **"The guidance doesn't cover this" is a first-class answer**, not a failure — the outcome that lets the third commitment hold without forcing a guess.

What that assures, precisely. The layer resolves versioned evidence for a stated jurisdiction, date and release, enforces citation, applicability, authority and known-gap checks on every claim, and records the evidence behind each response. It does not establish that an interpretation is correct — that still depends on the graph, the entry concepts chosen and faithful composition. The checks make the first kind of failure impossible and the others visible.

FOUR OUTCOMES, ONE CONTRACT

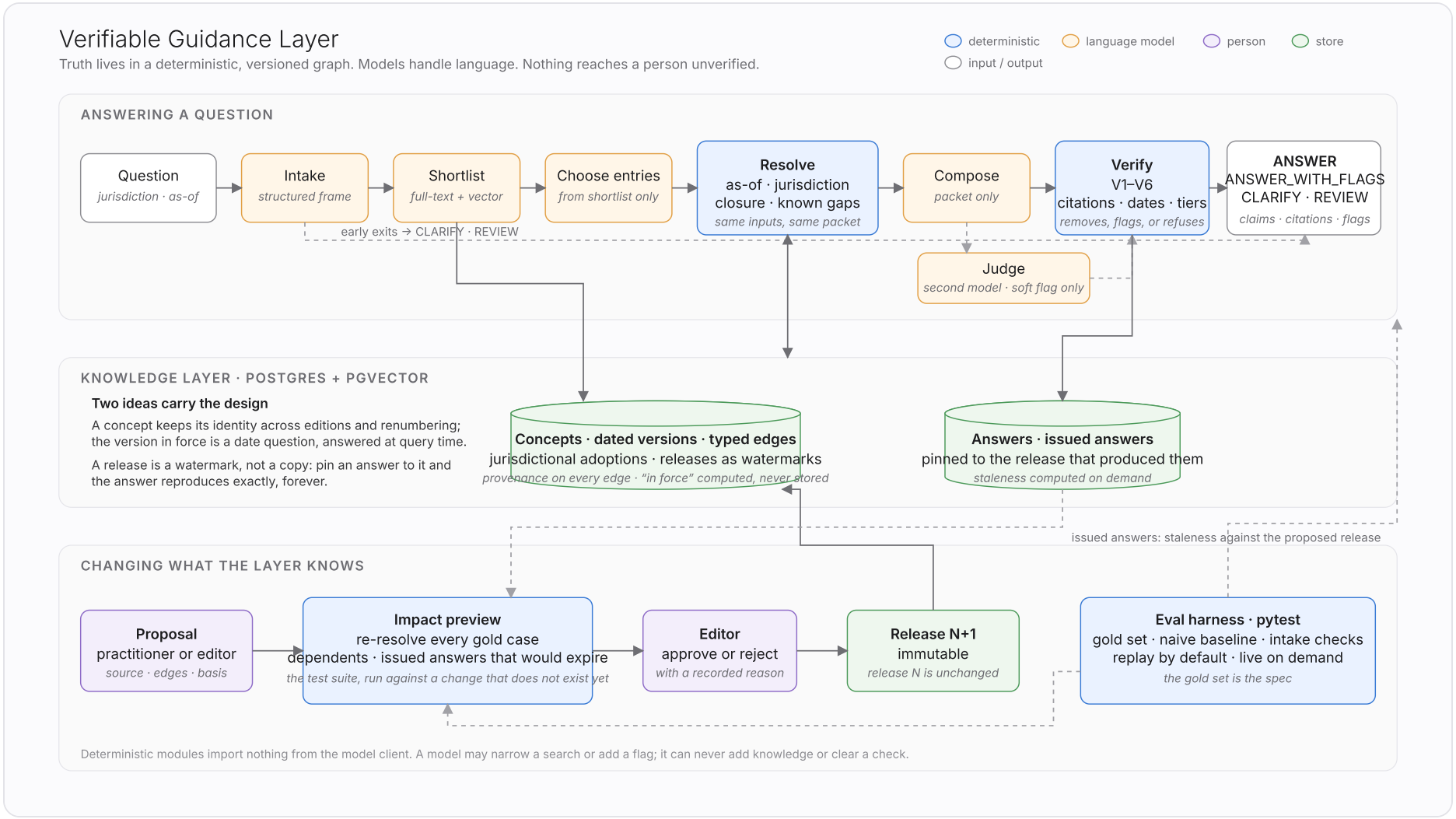
ANSWER	Every claim passed the declared checks; nothing outstanding. Not a proof that the interpretation is correct.
ANSWER with flags	The same, with a condition or notice attached — a carve-out, a pending adoption, an upcoming amendment, a practitioner note, a judge's note.
CLARIFY	A control (jurisdiction, as-of date) or a required case fact is missing. Ask the user.
REVIEW	A failed check, an unresolved reference, no coverage, or a conclusion that could not be grounded. Route to a person; name the gap.

The response carries the claims, each bound to the version IDs it rests on, the flags, the full evidence packet, the verification record and the release — the same JSON for a person's screen and for an agent's tool call. No path to an answer skips the checks; the one-paragraph summary is the composer's reading of the checked claims and is labelled as such.

Two failures the brief implies, also covered. Jurisdiction: per-jurisdiction adoption dates select different versions for the same date, with a pending-adoption flag. Time: releases, pinned answers, and computed staleness with a date and a reason.

The architecture

Nine stages to answer a question; one path to change what the layer knows. language model deterministic code



Resolve is two SQL statements — a bounded recursive walk and a per-concept in-force selection — and imports nothing from the model client. **Verify** is six rules in plain code: V1 every claim cites · V2 every citation is in the packet · V3 none is superseded · V4 every carve-out is surfaced · V5 an unresolved reference withholds finality · V6 practitioner material never grounds a claim. A bad supporting claim is removed and flagged; a bad conclusion fails the answer. The judge is a second, different model that can add a soft note and cannot clear a check.

The knowledge layer: four ideas, one Postgres

A CONCEPT KEEPS ITS IDENTITY

Editions amend *and renumber*. `prov:composite-inclusion-timing` is **3.A.4** in Edition 2021 and **3.A.6** in Edition 2025; the number a person sees is a per-version display reference, never an identity. Edges connect concepts, so an edge never needs re-asserting when an edition changes.

A RELEASE IS A WATERMARK, NOT A COPY

Every row records the release that created it and, if retired, the one that retired it. Adding a version in release N+1 closes the previous one *in N+1 only*; the evidence release N resolves reproduces byte for byte, forever, and the answer issued under it is stored. Answers pin to a release. Hard deletes are forbidden.

"IN FORCE" IS COMPUTED, NEVER STORED

A version carries only its effective date; a jurisdiction carries its own adoption date for each document. The version in force on a date is selected at query time: the latest jurisdiction-adjusted start on or before the as-of date. Historical and future as-of questions come for free, and no row is ever mutated when a successor arrives.

PROVENANCE ON EVERY EDGE

An edge is an assertion that two things relate, so each one records who asserted it, on what basis, from which passage, and who reviewed it. Nothing links two concepts without someone — or something — accountable for it.

TABLE	WHAT IT IS	WHAT IT ENABLES
`concepts`	A stable identity. A concept with no published version is a *known gap*	Renumbering; cross-edition citation; gaps that are detectable
`nodes`	A dated version: display reference, tier, obligation, text, machine-checkable parameters, effective date	As-of resolution; stale-citation detection (V3); upcoming-change flags
`edges`	Typed, concept-to-concept: `DEPENDS_ON_DEFINITION`, `REFERENCES`, `INTERPRETED_BY`, `CARVED_OUT_BY`, `ANNOTATED_BY` — with provenance	A changed definition reaches every provision that depends on it; carve-outs surface (V4); the reverse walk for impact previews
`releases`, `adoptions`	The watermark; a jurisdiction's effective date per document	Immutable history; the same question answered differently in two places
`issued_answers`, `proposals`	What was told to a client, pinned to the versions it relied on; bundles waiting for an editor	Staleness computed with a date and a reason; governance

Recorded output: a different place, and a gap the system knows about

SAME QUESTION, TWO JURISDICTIONS · RELEASE 1

By when must a new portfolio be added to a composite?

Jurisdiction North · as of 1 June 2025

ANSWER Under 3.A.6, a new portfolio must be included in a composite on a timely and consistent basis, no later than the beginning of the **first** full calendar month after it comes under management.

Jurisdiction South · as of 1 June 2025 — South adopts Edition 2025 on 1 January 2026

ANSWER with flags Under 3.A.4, a new portfolio must be included … no later than the beginning of the **second** full calendar month after it comes under management, unless the firm's composite-specific policy sets an earlier date.

> **Pending adoption here** — Edition 2025 is already in effect elsewhere but applies in JS only from 2026-01-01; it changes 3.A.4.

> **Practitioner note (JS)** — JS examiners expect the composite policy to state the specific inclusion month. Surfaced and labelled; can never ground a claim (V6).

naive RAG "first full calendar month", in both places. It has no representation of *where*.

THE SYSTEM LEARNS SOMETHING · RELEASE 1 → 2

A client withdrew 40% last month and the portfolio is now below our composite minimum. Must we remove it?

Jurisdiction North · as of 1 June 2025 · Release 1

REVIEW Generally yes, from the next measurement period it begins below the minimum — but a 40% withdrawal may engage significant-cash-flow treatment in Q&A QA-2025-031, which has not been reviewed.

3.A.7 says portfolios below the minimum must not be included; GS-2025-02 interprets it and refers to a ruling the system does not hold.

> **Referenced material not held** — QA-2025-031 is referenced by GS-2025-02 but is not held in this release. A *known* gap, named.

naive RAG Better than a hedge: it names the ruling and tells the reader to check it — the work the layer does, then routes.

A practitioner proposes the ruling: its text, three edges ("3.A.7 is carved out by QA-2025-031" and the two definitions it depends on), who asserted them and on what basis. Impact preview, before approval: **2 gold cases change, 0 issued answers affected.** Approve → Release 2.

Same question, same facts · Release 2

ANSWER with flags Not necessarily: under QA-2025-031 the portfolio may stay for up to three full measurement periods if the breach results solely from a significant cash flow and the documented policy provides for this; otherwise 3.A.7 requires removal.

> **Exception applies** — QA-2025-031 creates an exception to 3.A.7 that may change the conclusion.

Release 1 still says REVIEW. Its evidence reproduces exactly and its answer is kept — a system that learned something, not one that changed its mind.

Recorded output: the definition changes, the provision does not

THE AMENDMENT · RELEASE 2 → 3

The Board amends the definition of a *fee-paying portfolio* (1.D.6), effective 1 December 2026: portfolios of related parties with all fees waived in full are no longer fee-paying. **No provision in Section 3 changes a single word.**

Impact preview, before approval. The graph is walked backwards: `3.A.1` and `3.A.2` depend on 1.D.6. **Five gold cases change** — four gain a change notice, one flips. **One issued answer is affected:** on 10 March 2025 Harbor Point Advisors (ref. **HPA-77**) was told that fully fee-waived employee accounts are fee-paying and must be in a composite. Status after approval: *Expires 1 December 2026 — re-review before then.* The reason is recorded as learned after the answer was issued. It is detected because the answer *cites* the definition that changed; a new carve-out touching a cited provision without replacing a version would need the dependency walk the preview already does for concepts — the next step, not the current behaviour.

The provision never changed. The answer did. And the reason is something the system learned after the answer was issued — which is exactly the question a reviewer should ask of any such system: *did the regulation change, or did your system learn something it should already have known?* Scene B is the second; this is the first; the release history proves which.

THE SAME QUESTION EITHER SIDE OF THE DATE · RELEASE 3

Employee accounts with fully waived fees — must they be in a composite?

Jurisdiction North · as of 15 November 2026

ANSWER with flags — Fully waived employee accounts still count as fee-paying under 1.D.6 if they are subject to the firm's fee schedule, and must then be in a composite if discretionary under 3.A.1.

> **Changes soon** — 1.D.6 changes on 2026-12-01 (Amendment 2025-A1). This answer applies before that date.

Jurisdiction North · as of 15 December 2026

ANSWER with flags — Not required if the employees count as related parties: fully waived accounts of related parties are not fee-paying under 1.D.6 as amended, so 3.A.1's mandatory inclusion does not apply — though they may be included, with disclosure.

Cited: `def:fee-paying-portfolio@E2025` before the date, `…@E2025-A1` after. Same provision text both times; the resolver selected a different definition version, and the verifier would have removed a citation to the wrong one.

naive RAG — Answers "no" in November too — it applies the amendment sixteen days before it is in force, and cannot say so.

The evidence, generated by the test suite

13 / 13
gold cases pass — outcome, citations, flags and gaps, never prose equality

100%
composed claims passed the structural checks; expected evidence present; modifiers surfaced

0 / 70
composed citations to a superseded version — the packet never offers one; after V3 the rate is 0 by construction

59
tests in about two seconds; 28 are unit tests of the deterministic core with no model anywhere

THE BASELINE, HONESTLY

Ordinary document RAG over the *same* prose with the *same* composer model — given the **question only**, no date and no place, as usually deployed. The tests record what it did and classify it by a simple rubric (superseded citation, wrong month, forbidden conclusion); they do not assert that it fails.

It gets **10 of 13** right, two traps included. It fails the three where the right answer is not the newest text applied everywhere, now:

CASE	WHY IT IS WRONG
G2 a different place	"first full month" in Jurisdiction South, where Edition 2025 is not yet in force
G7 a past date	"first full month" as of June 2024, before Edition 2025 was even published
G11a an amendment not yet in force	applies the December amendment to a November question

And it cannot tell the user which of its answers those are. They are failures of context-blind retrieval, not of the model: a baseline given the date and place in its prompt is the obvious next comparison, not yet run. On these thirteen cases the layer's advantage is **zero unflagged wrong answers, and knowing when not to answer.**

WHAT EACH NUMBER IS

METRIC	VALUE
Gold pass rate (layer)	100% (13/13)
Structural validation rate (claims passing V1–V6 ÷ composed)	100%
Citations to a superseded version	0 of 70 composed; 0 after V3, by construction
Gold-evidence coverage (expected versions in packet ÷ expected)	100%
Modifier surfaced rate	100%
Abstention precision / recall	100% / 100%
Intake (no invented date, place or fact)	5 / 5
Judge disagreements (soft notes)	3 — never change a verdict
Baseline failure rate (question only; rubric)	23% (3 of 13)
Semantic correctness, independently assessed	not measured

On thirteen hand-authored cases, 100% is what it should be: every trap set was caught, nothing more.

Three decisions that were not obvious — and what comes next

1

NO GRAPH DATABASE

The two graph operations the product needs — follow the required relationships for this case; what depends on this changed clause — are shallow (≤ 4 hops) and bounded. A recursive CTE over an indexed edge table does them in milliseconds, and one Postgres holds text, vectors, edges, releases and audit. The schema already *is* a property graph; the export is mechanical when whole-graph pattern queries or graph algorithms are needed.

2

"IN FORCE" COMPUTED, NEVER STORED

Stored end dates are the classic source of silent inconsistency, and updating them when a successor arrives mutates rows that earlier releases depend on. Computing in-force at query time is what makes a release a watermark instead of a copy, and what makes "open the release this answer was given under and it reproduces" true rather than aspirational. The cost: effective dates must be right at ingestion.

3

VERIFICATION AS A SEPARATE DETERMINISTIC PASS

The brief's failure is confident wrong answers; a gate that is itself probabilistic cannot be the answer to that. Six rules in plain code, testable without the thing they gate, with a graded contract: a bad supporting claim is removed and flagged, a bad conclusion fails the answer. The judge exists, as a second model whose only power is to add a soft note.

AN HONEST LIMITATION

The layer makes errors **visible and attributable**, not impossible. A wrong edge or a wrong effective date produces a well-cited wrong answer. The verifier checks the answer against the graph; it cannot check the graph against the world. The defences are provenance, editorial review, the impact preview, and the gold set.

WITH MORE TIME, IN ORDER

1. **Extraction** through the same proposal path — then one real public-domain document through it.
2. **Judge meta-evaluation**: human labels on claim/citation pairs, so the judge's own precision and recall are known.
3. **Composer-sensitivity matrix** across models: correctness from the layer, fluency from the model.
4. Jurisdictional overrides, edge validity windows, true bitemporal "known since"; editorial roles and an audit view; the answer contract as an agent tool.

How to review this

TEN MINUTES, NOTHING TO INSTALL

ericediger.com/vgl — the walkthrough, in nine sections. Press → or click to advance; **f** opens the full recorded answer behind any frame; **?** shows the keys. Everything on it is recorded output of the system; the only thing simulated is the latency.

FIFTEEN MINUTES, DOCKER ONLY

```
git clone https://github.com/ericediger/verifiable-guidance-layer.git
cd verifiable-guidance-layer
VGL_LLM_MODE=replay docker compose up --build
```

Open `http://localhost:8000`. The header shows *Release 1 · 40 versions* and a *recorded responses* tag. Click any question chip — the thirteen gold cases — and run both scenes from the Releases tab: impact preview → approve → ask again on the new release; *Issued answers* shows the client answer that expires. **Reset to Release 1** is at the bottom of the Releases tab.

With your own keys in `.env` and `docker compose up --build`, ask it anything. The model steps run the same code path either way.

```
docker compose run --rm app pytest     # 59 tests, ~2 s, no network
```

READ, IN THIS ORDER

01 · Architecture	The design, the mechanisms for the brief's three failures, the 0 → 1 path.
03 · Decisions	Fourteen non-obvious decisions, ADR-style; what the first live run changed.
Eval report	Generated by `pytest`: metrics, every case, the baseline verbatim.
04 · Demonstration	What is recorded, what runs, how it was built (appendix overleaf).
00 · Corpus spec	The fictional corpus and the gold set; committed before any code.

WHERE TO LOOK FIRST IN THE CODE

`src/vgl/closure.py`	The resolver: a bounded recursive walk, then in-force selection. No model import.
`src/vgl/verify.py`	Rules V1–V6 and the graded failure contract; twelve unit tests.
`src/vgl/releases.py`	The impact preview: load the bundle in a savepoint, re-resolve, roll back.
`src/vgl/schema.sql`	The data model, with the two ideas written at the top.
`data/gold.yaml`	Correct behaviour, as data.

Appendix: how the demonstration was built

WHAT THE HOSTED WALKTHROUGH IS

A static, keyboard-driven presentation of **recorded output of the real system**. On 4 November 2026 the full pipeline was run live — Opus composing, Sonnet judging and parsing intake, OpenAI embeddings — against Releases 1, 2 and 3, and the results saved as JSON. The site renders those files; nothing on it calls a model or a database. **Simulated: only the latency** (the Ask progress bar is a 2.5-second animation; live, a question takes 10–25 seconds) and the moment the release badge bumps. Every verdict, claim, citation, flag, judge note and number is the system's actual output.

WHAT RUNS IN THE REPOSITORY

	REPLAY (NO KEYS)	LIVE (KEYS)
Resolve, verify, releases, impact preview, staleness	Real	Real
Intake, choose, compose, judge, embeddings	Recorded responses, one code path	Real API calls
The thirteen gold questions; both scenes; reset	Work	Work
Any other question	Plain 409 message	Works
`pytest`	Default	`--live`, `--repeat N`

HOW IT WAS BUILT

1. **Spec first.** The corpus, the traps, the outcome contract and the gold set were written and committed before any code.
2. **The gold set is the behavioural spec**, as data; evals assert structure, never prose.
3. **The deterministic core with unit tests before any model call**; `closure.py` and `verify.py` import nothing from the model client.
4. **Record/replay from day one**; the suite runs with no network.
5. **The baseline recorded, never asserted.**
6. **The UI last**, over the same JSON an agent receives.

The structured graph was authored by hand to match the prose, with provenance recorded on every edge as if produced by an extractor and reviewed by an editor; extraction at scale was deliberately kept out of a proof of concept. The code was written by an AI coding agent (Claude Code) under the author's direction and a written working agreement — nothing ships that cannot be explained in two sentences; no real regulation; provenance on every edge; commit messages that state a decision. The architecture, the corpus, the decisions and the scope cuts are the author's.

A PUBLIC LIVE DEMO WOULD NEED

A hosted Postgres 16 with pgvector; the FastAPI container behind HTTPS with the two keys as platform secrets; a spend cap in both vendor consoles (each live question is three to four model calls, 10–25 s on the 4 November run, and a few cents); rate limits and a passcode in front of `/api/ask`; a nightly reset to Release 1. About one working day; the code needs no change beyond configuration.